Task 3: Infrastructure as Code (IaC) with Terraform

Objective

Provision a local Docker container using Terraform.

Tools Used

* Terraform
* Docker Desktop
* VS Code
* Git Bash
* Nginx Docker Image

Project Description

In this task, I used Terraform as an Infrastructure as Code (IaC) tool to provision a Docker container locally.

I used the Terraform Docker provider to create an Nginx Docker image and an Nginx container.

The container was named:

```text
terraform-nginx
```

The container port was mapped as:

```text
localhost:8081 → container port 80
```

Port 8081 was used because port 8080 was already being used by Jenkins.

Project Structure

```text
terraform-docker/
│
├── main.tf
├── README.md
├── .gitignore
└── screenshots/
    ├── 01-terraform-init.png
    ├── 02-terraform-validate.png
    ├── 03-terraform-plan.png
    ├── 04-terraform-apply.png
    ├── 05-docker-ps.png
    ├── 06-terraform-state.png
    └── 07-terraform-destroy.png
```

Terraform Configuration

The `main.tf` file contains the Terraform configuration for the Docker provider, Nginx image, and Nginx container.

```hcl
```

```
terraform {
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}

provider "docker" {}

resource "docker_image" "nginx" {
  name = "nginx:latest"
}

resource "docker_container" "nginx" {
  name  = "terraform-nginx"
  image = docker_image.nginx.image_id

  ports {
    internal = 80
    external = 8081
  }
}
```

Implementation Steps

1. Terraform Initialization

Command:

```bash
terraform init
```

Terraform successfully initialized the project and installed the Docker provider.

Result:

```text
Terraform has been successfully initialized!
```

Screenshot

![Terraform Init](screenshots/01-terraform-init.png)

---

2. Terraform Validation

Command:

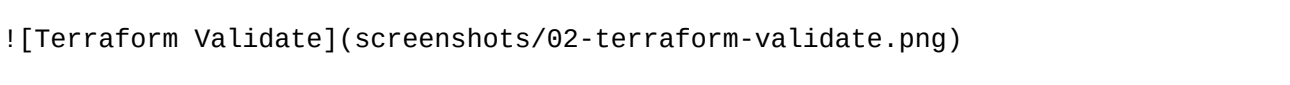
```bash
terraform validate
```

The Terraform configuration was validated successfully.

Expected result:

```text
Success! The configuration is valid.
```

Screenshot



---

3. Terraform Plan

Command:

```bash
terraform plan
```

Terraform created an execution plan before making changes.

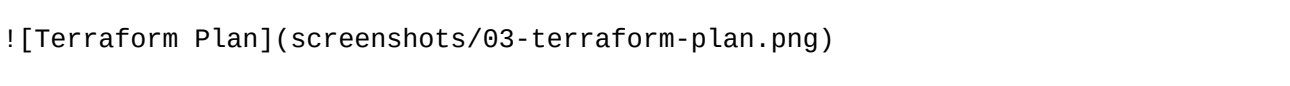
Result:

```text
Plan: 2 to add, 0 to change, 0 to destroy.
```

The two resources were:

* Docker Nginx image
* Docker Nginx container

Screenshot



---

4. Terraform Apply

Command:

```bash
terraform apply
```

I confirmed the operation by entering:

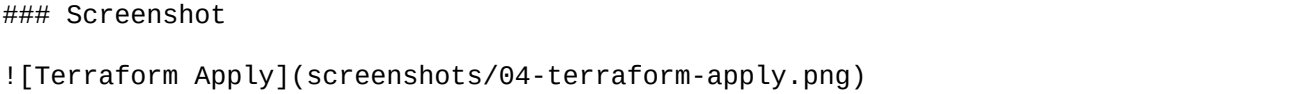
```text
yes
```

Result:

```text
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```

Terraform successfully created the Nginx Docker image and container.

Screenshot



---

5. Verify Docker Container

Command:

```bash
docker ps
```

The Terraform-created container was running:

```text
terraform-nginx
```

Port mapping:

```text
0.0.0.0:8081 → 80/tcp
```

The Nginx application could be accessed at:

```text
http://localhost:8081
```

Screenshot

![Docker PS](screenshots/05-docker-ps.png)

---

6. Check Terraform State

Command:

```bash
terraform state list
```

Terraform was tracking the following resources:

```text
docker_container.nginx
docker_image.nginx
```

This confirmed that Terraform was managing the Docker infrastructure.

Screenshot

![Terraform State](screenshots/06-terraform-state.png)

---

7. Destroy Infrastructure

Command:

```bash
terraform destroy
```

I confirmed the operation by entering:

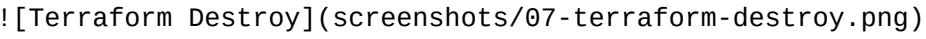
```text
yes
```

Result:

```text
Destroy complete! Resources: 2 destroyed.
```

The Terraform-managed Nginx container and image were successfully removed.

Screenshot



Execution Summary

Command	Result
`terraform init`	Successful
`terraform validate`	Successful
`terraform plan`	2 resources to add
`terraform apply`	2 resources created
`docker ps`	`terraform-nginx` running
`terraform state list`	2 resources tracked
`terraform destroy`	2 resources destroyed

Outcome

This task helped me understand the basic Infrastructure as Code lifecycle using Terraform.

I learned how to:

* Write Terraform configuration
* Use the Docker provider
* Initialize Terraform
* Validate Terraform configuration
* Create an execution plan
* Provision Docker infrastructure
* Verify Docker resources
* Check Terraform state
* Destroy infrastructure using Terraform

Final Result

The local Nginx Docker container was successfully provisioned and managed using Terraform.

The complete Terraform lifecycle was successfully performed:

```text
Write Configuration
       ↓
terraform init
       ↓
```

```
terraform validate
       ↓
terraform plan
       ↓
terraform apply
       ↓
Verify Docker
       ↓
terraform state list
       ↓
terraform destroy
```

Completed Successfully.